life.augmented

基于STM32G4电机控制进阶篇

意法半导体微控制器部门

Agenda

1. STM32G4内部资源利于电机控制
2. 电流环放在STM32G4 CCM SRAM中
3. 内部电压参考作为ADC基准软件修改
4. 电流采样使用STM32G4内部运放
5. STM32G4的32-bit Timer应用于编码器电机应用

STM32G4内部资源利于电机控制

life.augmented

MC SDK 5.x中STM32G4资源使用

- 三电阻采样使用的资源

电机	Adv.Timer	ADC	DMA	NVIC	Other
单电机	TIM1	ADC1/ADC2	NA	ADC1_2， TIM1_BRK TIM1_UP， SysTick	Cordic
双电机	TIM1/TIM8	ADC1/ADC2 ADC3/ADC4	NA	ADC1_2， TIM1_BRK TIM1_UP， SysTick ADC3，TIM8_BRK TIM8_UP	Cordic

- 单电阻采样使用的资源

电机	Adv.Timer	ADC	DMA	NVIC	Other
单电机	TIM1	ADC1(或ADC2)	TIM1_CH4	ADC1_2， TIM1_BRK TIM1_UP， SysTick	Cordic
双电机	TIM1/TIM8	ADC1,2,3,4任选其二	TIM1_CH4 TIM8_CH4	ADC1_2， TIM1_BRK TIM1_UP， SysTick (ADC3/4)，TIM8_BRK TIM8_UP	Cordic

STM32G4助力电机控制

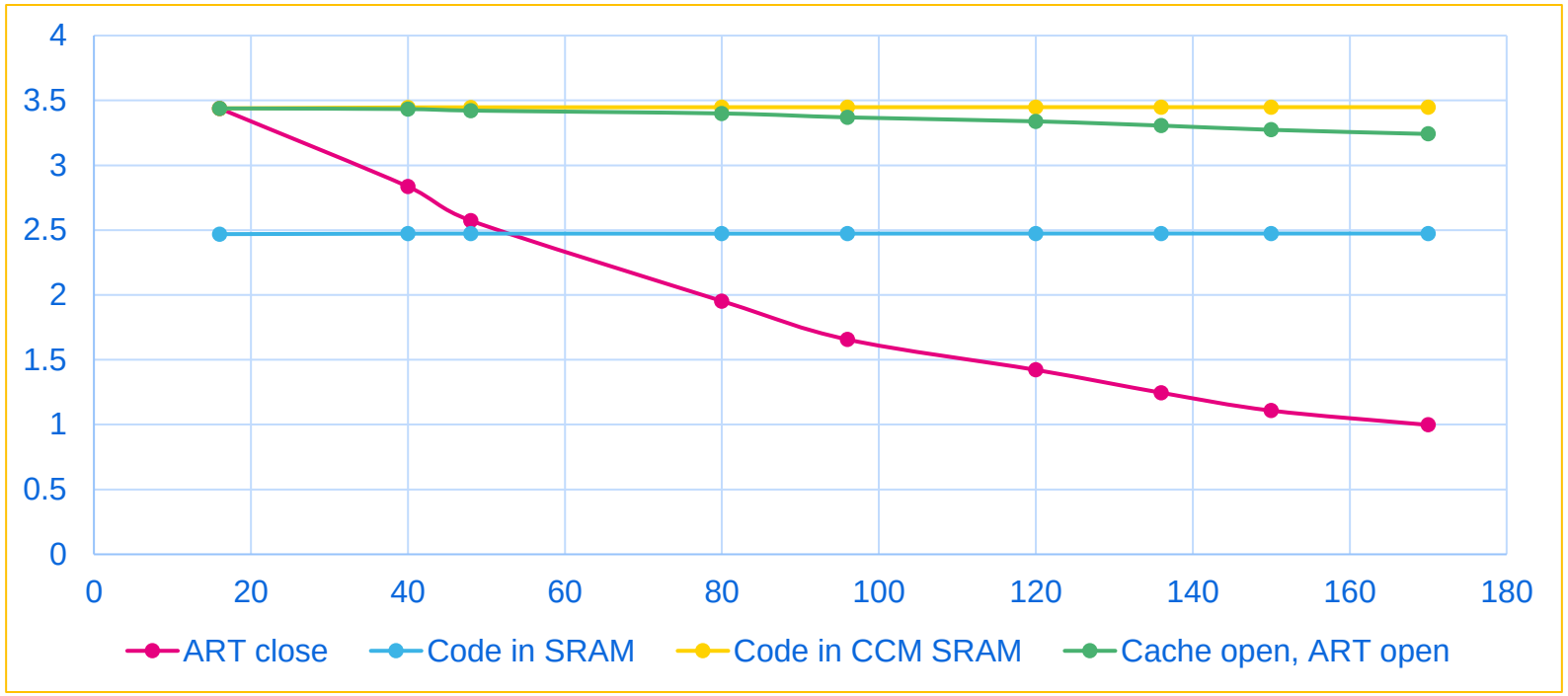
CCM-SRAM	• 核心程序执行，达到最高执行速率
浮点运算	• 单精度，完成复杂算法（如高频注入）
Dithering	• 20-bit PWM控制精度
编码器模式	• 支持更多，更灵活的模式
VREFBUF	• 内部精准参考电压
运放,比较器	• 成本，板材面积
Cordic	• 更快的计算，更快的环路

电流环放在STM32G4 CCM SRAM中

程序运行在CCM SRAM

- 程序在CCM SRAM中才能完美发挥出STM32G4的性能
- MC SDK V5.4.3中STM32G4默认并未使用CCM SRAM

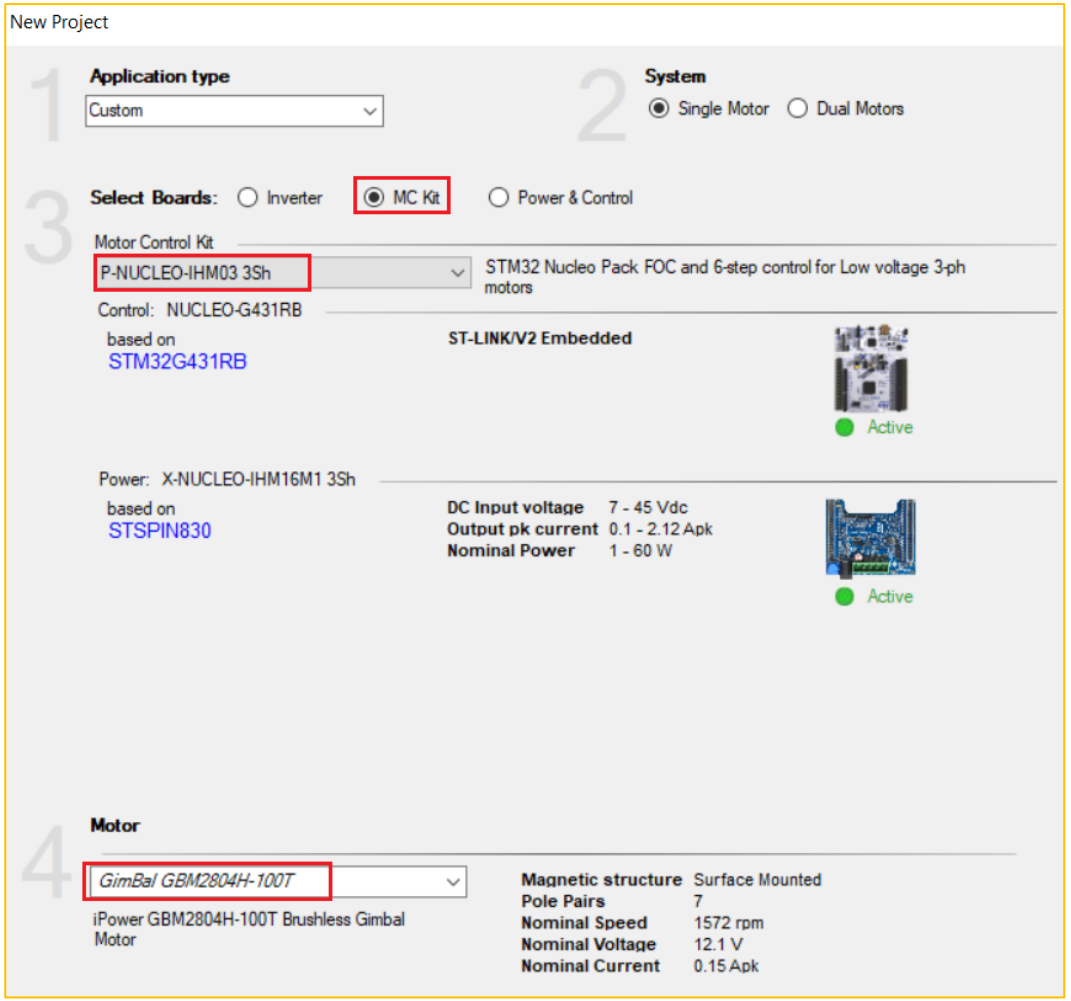
Cormark



life.augmented

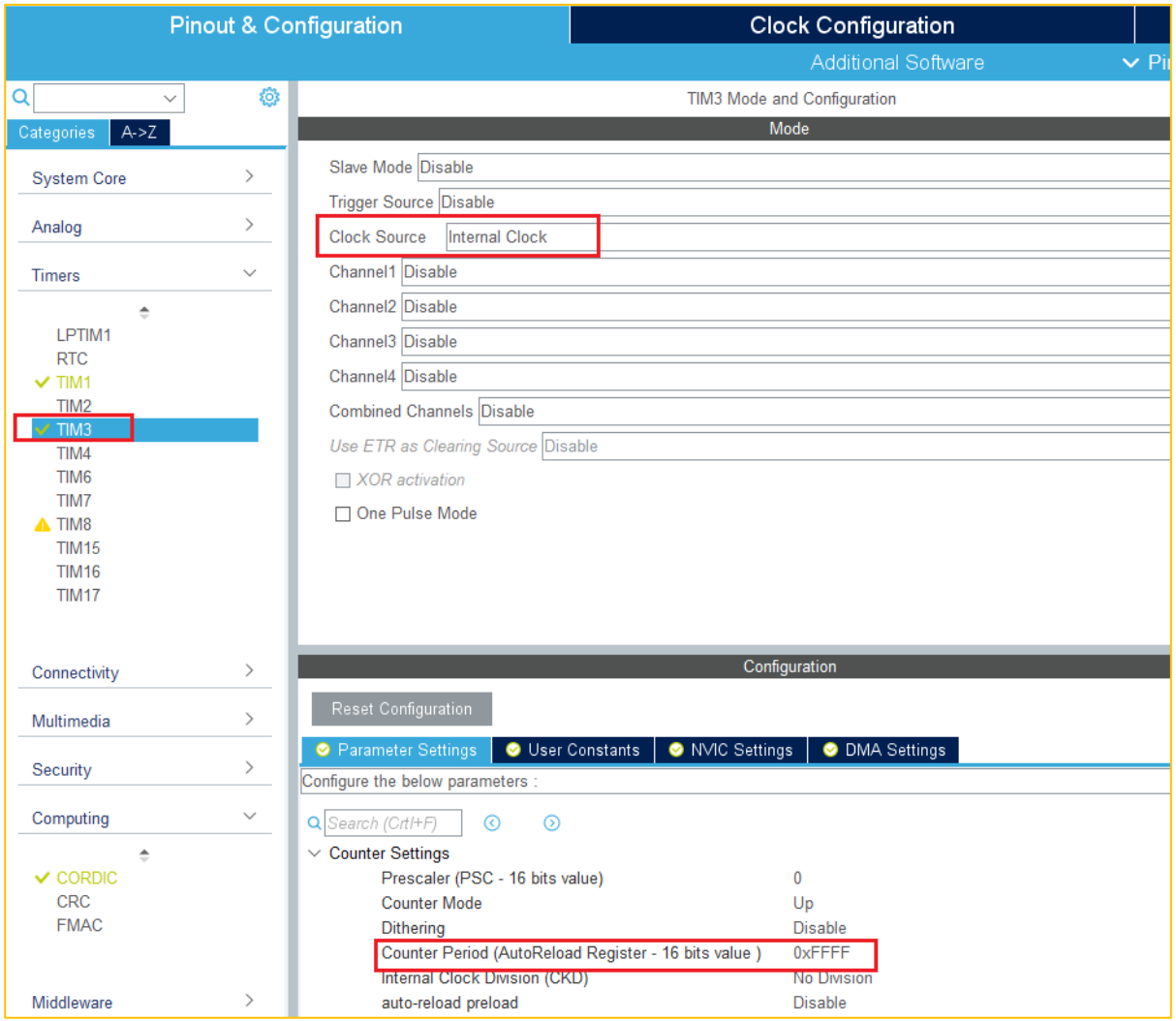
生成P-Nucleo-IHM03电机套件工程

- 按照培训基础篇章介绍生成工程



使能TIM3计数用于电流环时间测试

- 打开CubeMx工程
- 使能并配置TIM3
- 最大溢出时间设定为
  - 65536/170MHz = 385.5us



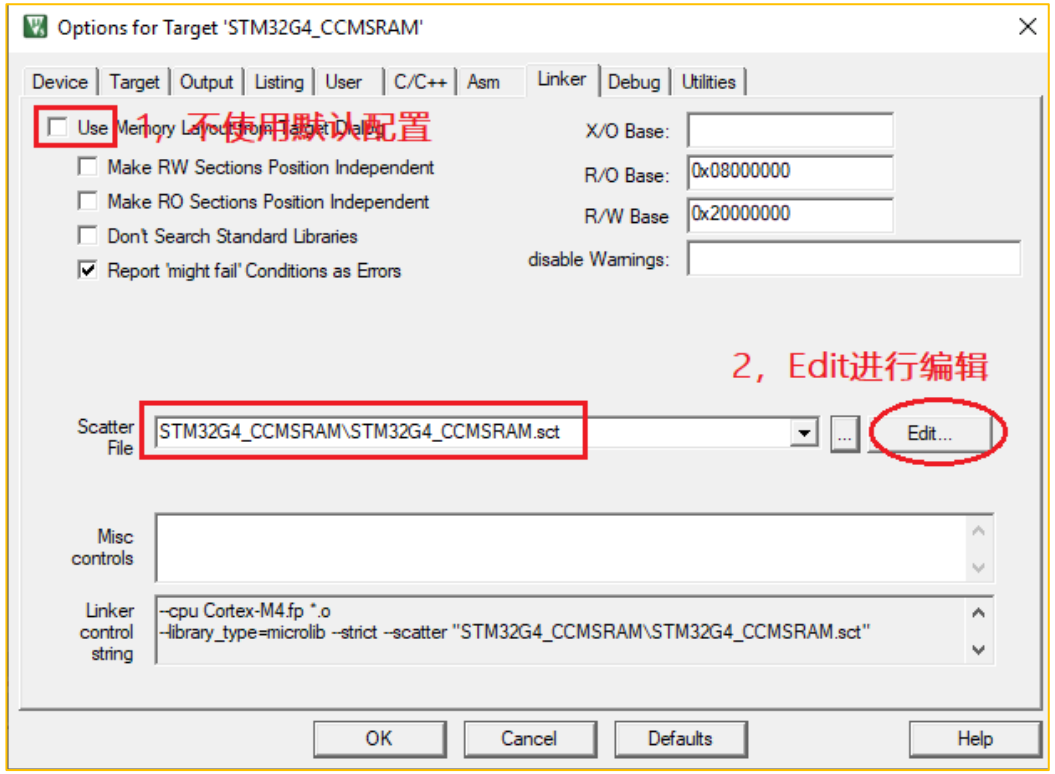
电流环中加入测试代码

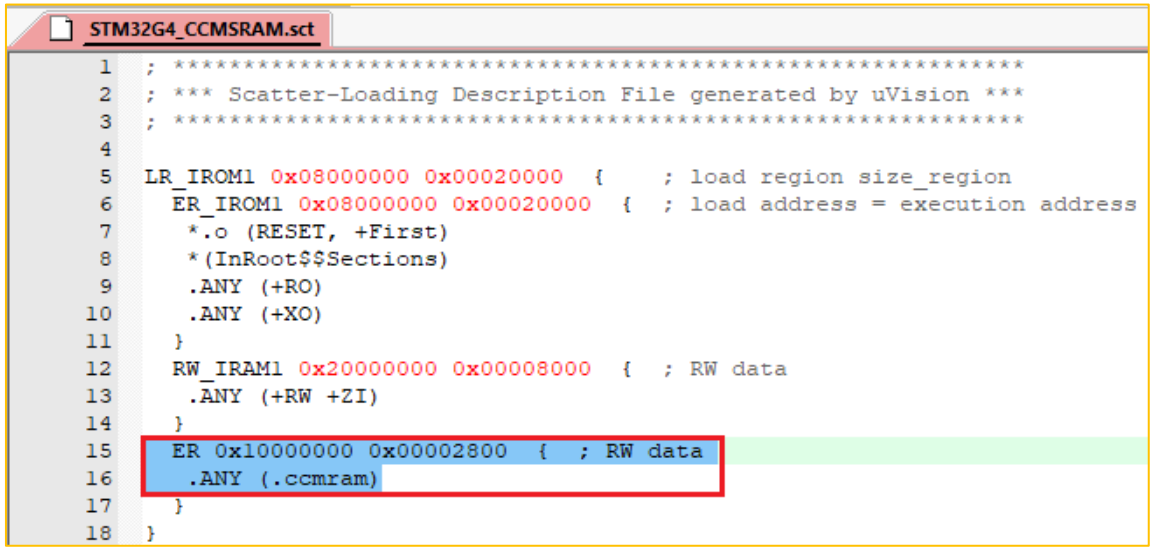
- 电流环在ADC中断ADC1_2_IRQHandler中
- TIM3从0计数，电流环运行结束后停止计数
- 函数在stm32g4xx_mc_it.c中

```
void ADC1_2_IRQHandler(void)
{
  /* USER CODE BEGIN ADC1_2_IRQn 0 */

  volatile static uint16_t FOC_Cycle;
  volatile static uint32_t FOC_Time;
  TIM3->CNT = 0;
  TIM3->CR1 |= 0x01;

  /* USER CODE END ADC1_2_IRQn 0 */

  // Clear Flags M1
  LL_ADC_ClearFlag_JEOS( ADC1 );

  // Highfrequency task
  UI_DACUpdate(TSK_HighFrequencyTask());
 /* USER CODE BEGIN HighFreq */

 /* USER CODE END HighFreq  */

 /* USER CODE BEGIN ADC1_2_IRQn 1 */

  TIM3->CR1 &= 0xFFFE;
  FOC_Cycle = TIM3->CNT;
  /* Time unit in us */
  FOC_Time = (uint32_t)FOC_Cycle/170;

 /* USER CODE END ADC1_2_IRQn 1 */
}
```

IDE修改—Keil MDK(1/3)

- 在Keil的Link File中添加CCM SRAM地址以及空间大小
- 第一次编译下然后才有sct文件。

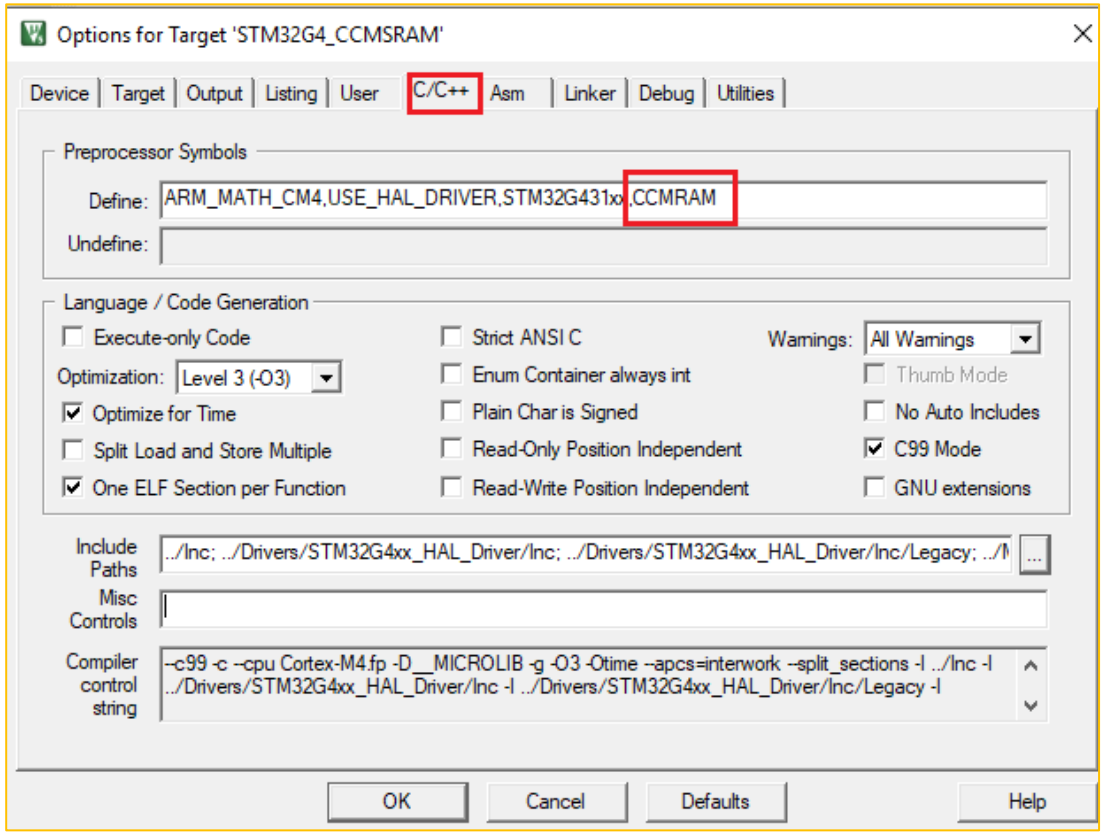


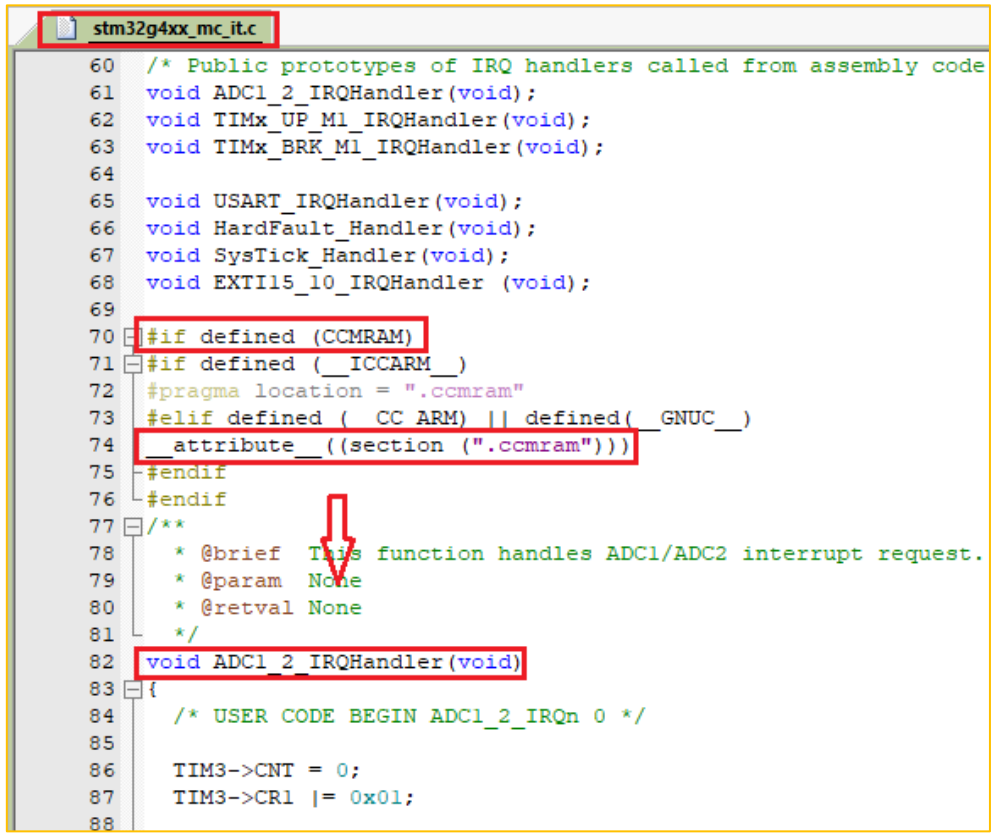


life.augmented

IDE修改—Keil MDK(2/3)

- 定义使用CCM SRAM

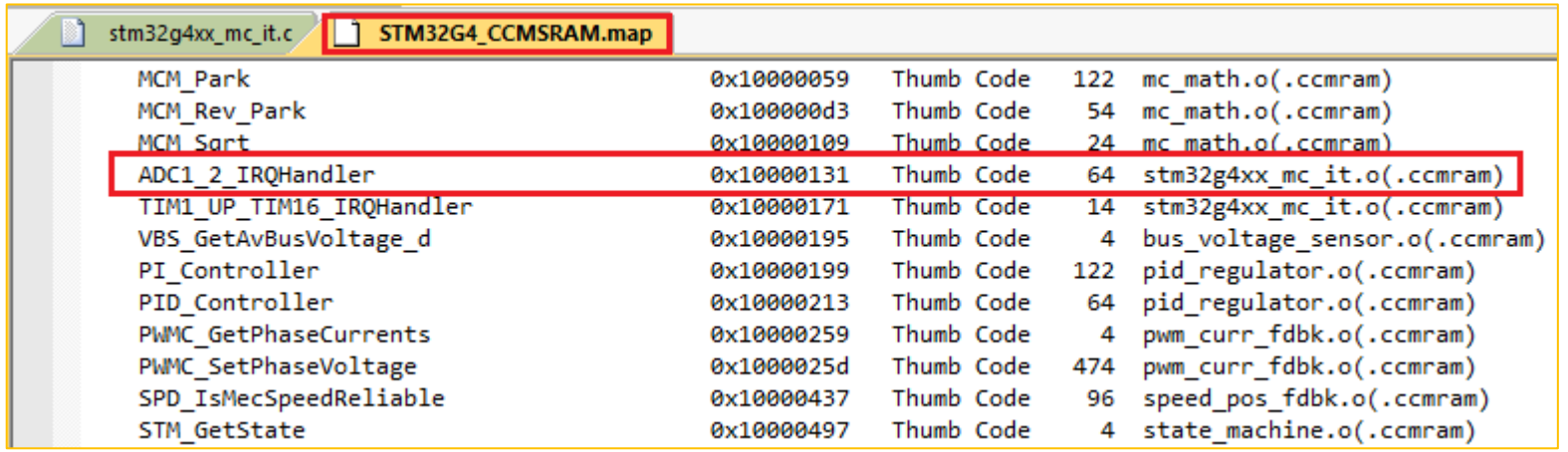




life.augmented

IDE修改—Keil MDK(3/3)

- 编译工程后打开.map文件
- 可以看到ADC1_2_IRQHandler执行地址在CCM SRAM中



life.augmented

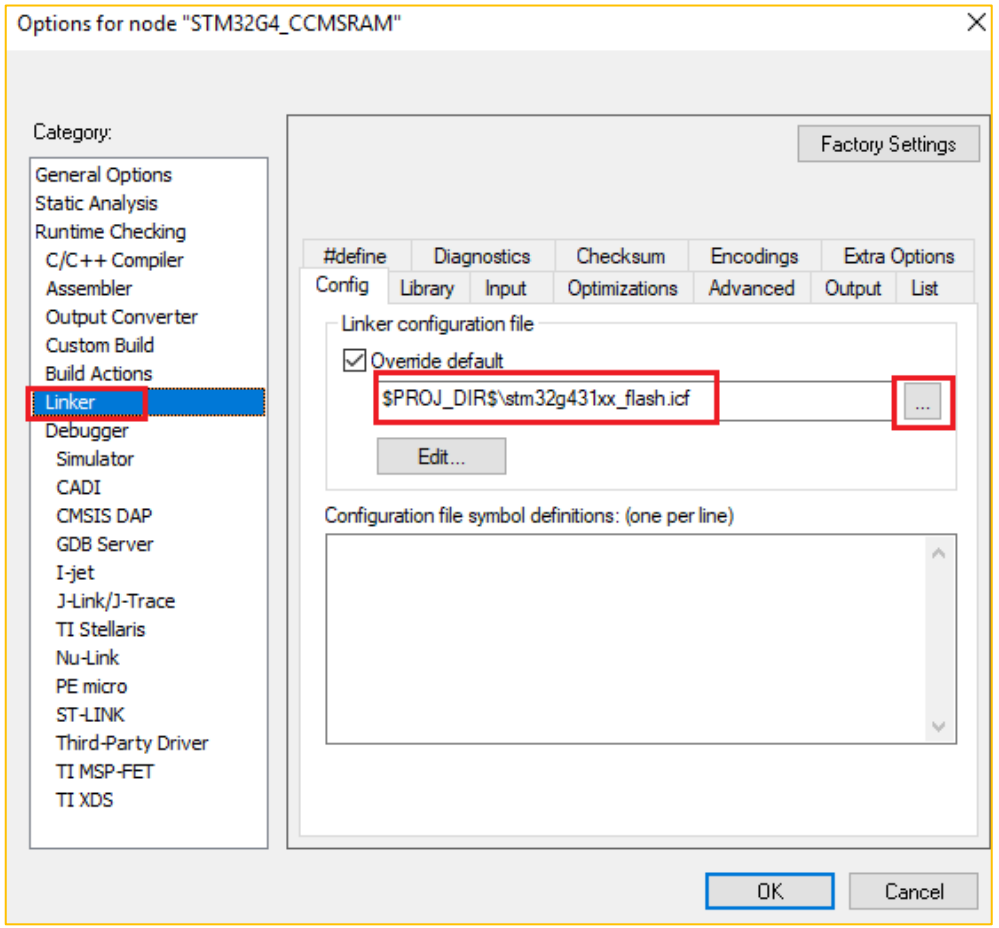
对比电流环执行时间

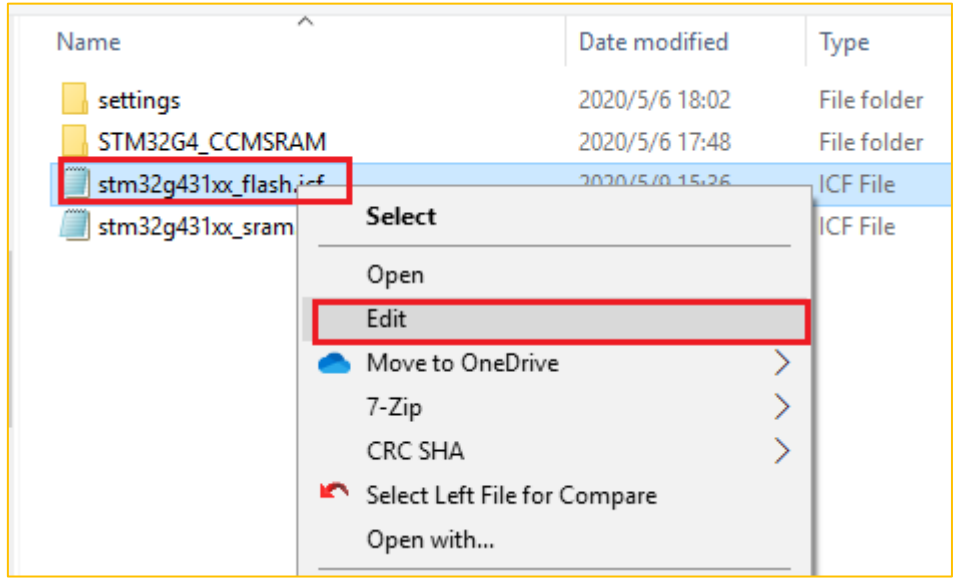
- 未使用CCM SRAM时，电流环执行时间为17us
- 使用CCM SRAM时，电流环执行时间为11us！！

life.augmented

IDE修改—IAR(1/6)

- 使用IAR时，打开并修改Link file文件*.icf



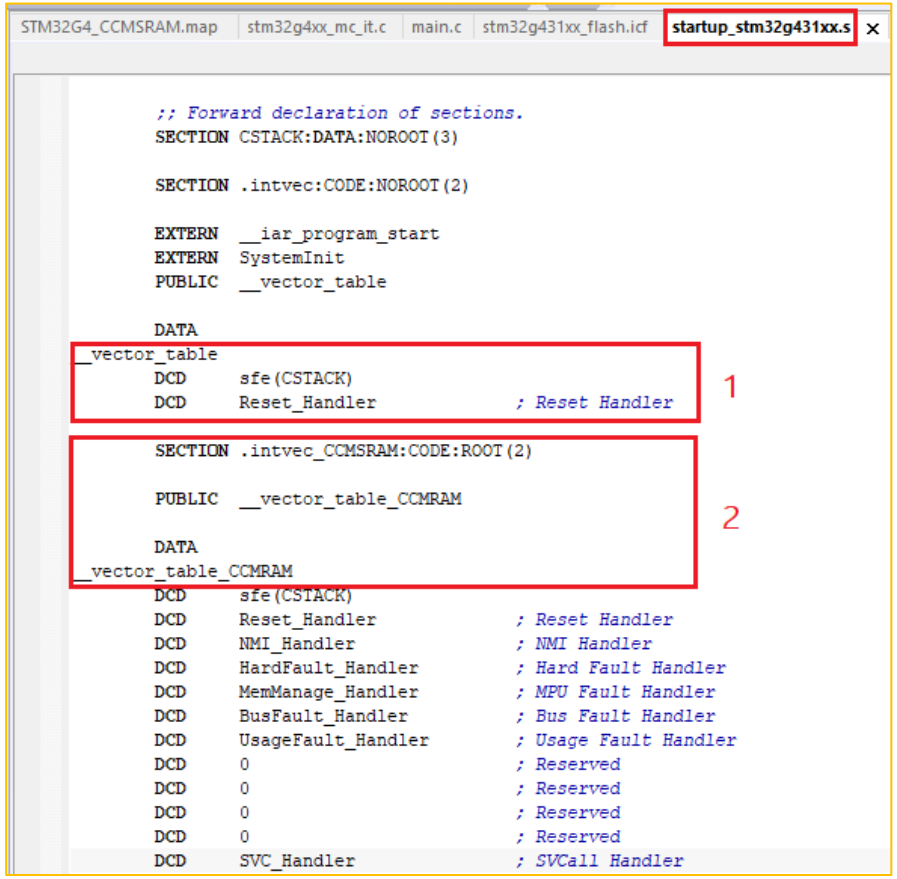


IDE修改—IAR(2/6)

- 修改icf文件
- 定义关键字用于函数放置段配置

IDE修改—IAR(3/6)

- 修改启动文件startup_stm32g431xx.s
- 添加需放入CCM SRAM中的中断向量表，该向量表放在修改过的icf文件的.intvec_CCMSRAM中



IDE修改—IAR(4/6)

- 将向量表重新映射至CCM SRAM
- 修改system_stm32g4xx.c文件中的SystemInit函数

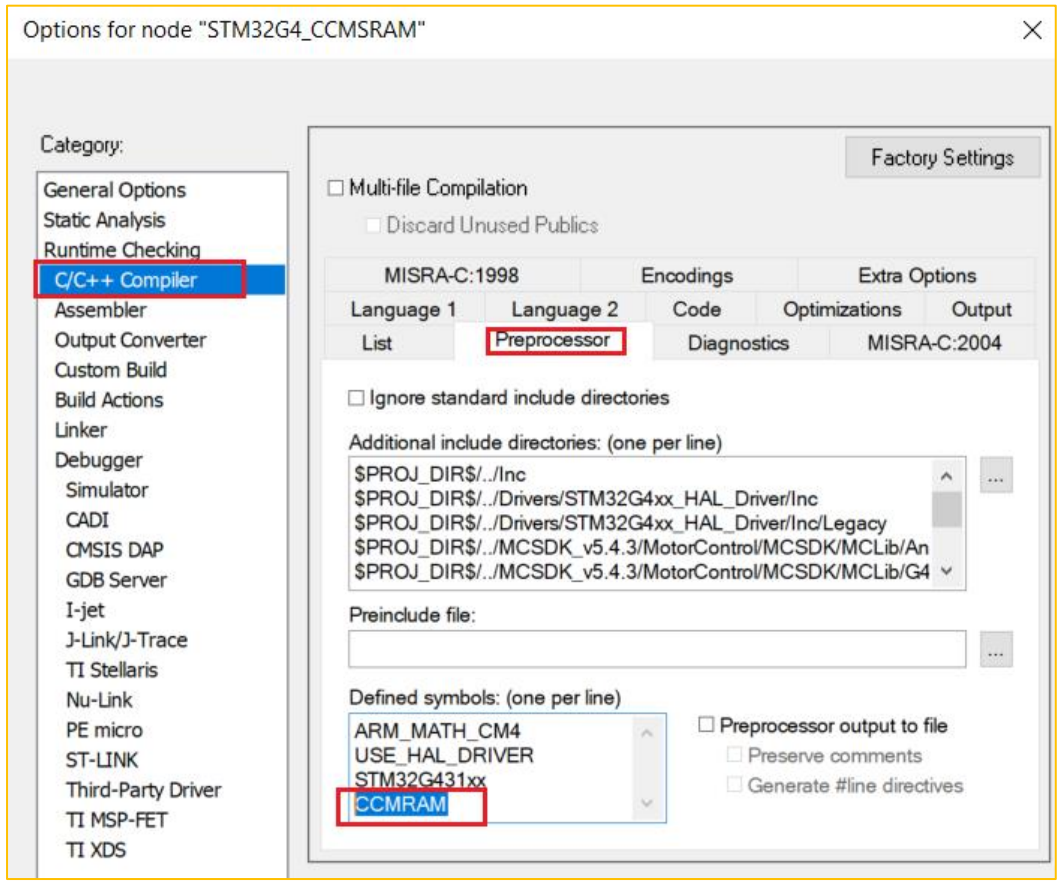
system_stm32g4xx.c

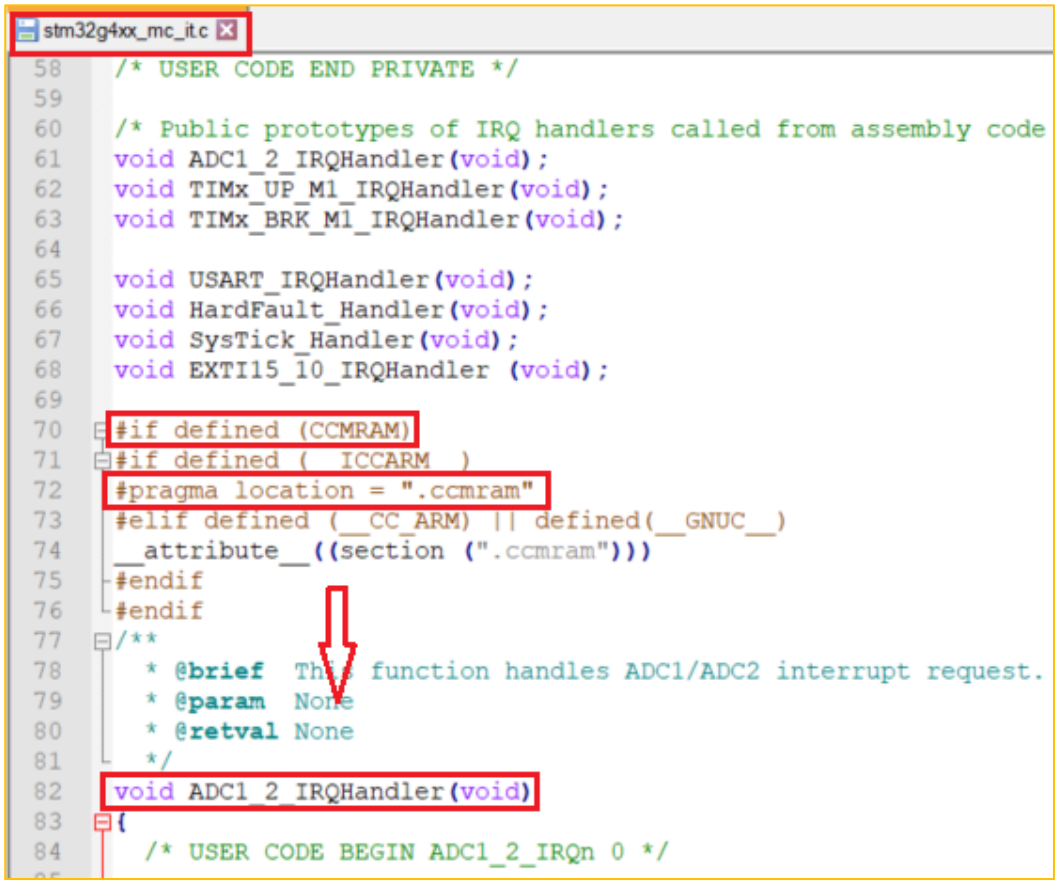
```
  * @brief  Setup the microcontroller system.
  * @param  None
  * @retval None
  */

void SystemInit(void)
{
  /* FPU settings ------------------------------------------------------------*/
  #if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
    SCB->CPACR |= ((3UL << (10*2))|(3UL << (11*2)));  /* set CP10 and CP11 Full Access */
  #endif

  /* Configure the Vector Table location add offset address ------------------*/
#ifdef VECT_TAB_SRAM
  SCB->VTOR = SRAM_BASE | VECT_TAB_OFFSET; /* Vector Table Relocation in Internal SRAM */
#else
//  SCB->VTOR = FLASH_BASE | VECT_TAB_OFFSET; /* Vector Table Relocation in Internal FLASH */
  SCB->VTOR = 0x10000000 | VECT_TAB_OFFSET;
#endif
}
```

life.augmented

IDE修改—IAR(5/6)

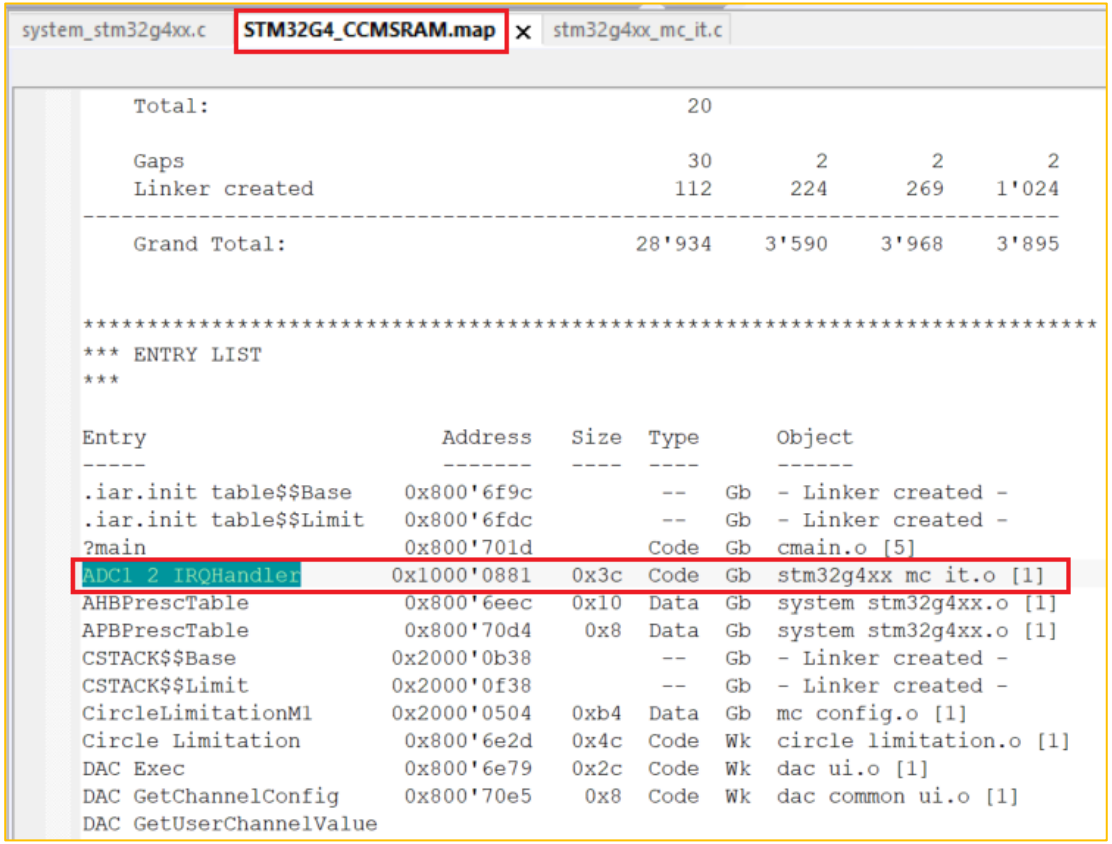
- 定义使用CCM SRAM





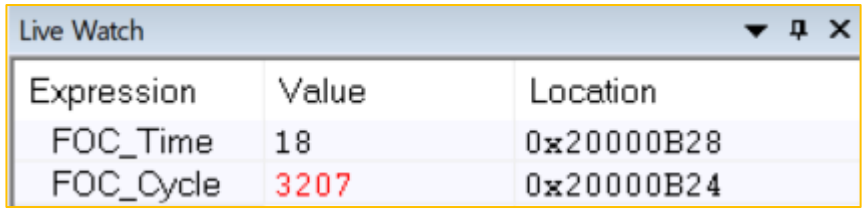
IDE修改—IAR(6/6)

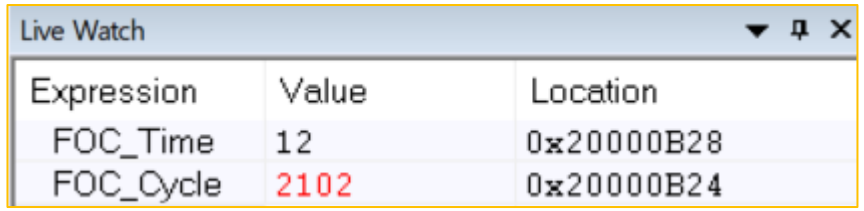
- 编译工程后打开.map文件
- 可以看到ADC1_2_IRQHandler执行地址在CCM SRAM中



对比电流环执行时间

- 未使用CCM SRAM时，电流环执行时间为18us
- 使用CCM SRAM时，电流环执行时间为12us！！

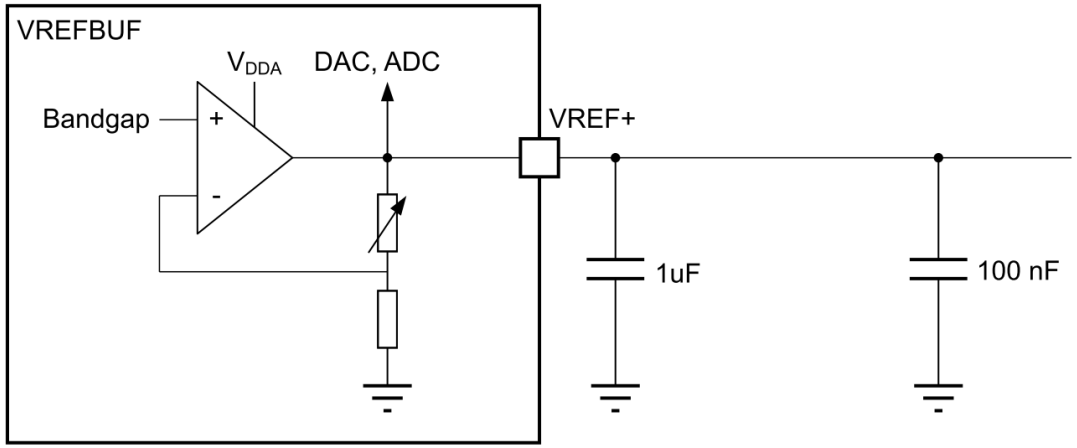




内部电压参考作为ADC基准软件修改

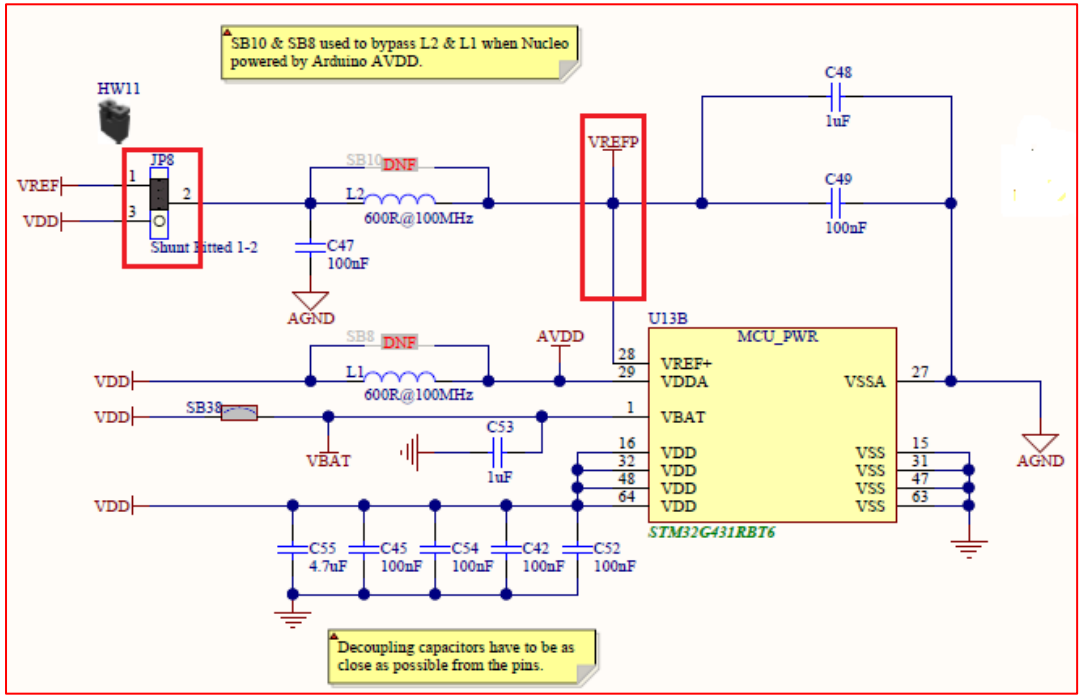
内部精准电压参考

- 内部电压参考，并可输出到VREF+ 管脚
  - 为ADC、DAC提供电压电压参考
  - 可输出到VREF+ 管脚，提供外部使用
- 3种电压等级:
  - ~2.048 V … (VDDA>2.40V)
  - ~2.500 V … (VDDA>2.80V)
  - ~2.900 V … (VDDA>3.135V)
- 软件操作举例，配置为2.5V电压参考
  - VREFBUF->CSR = 0x11;



硬件准备

- 本例采用P-NUCLEO-IHM03为测试硬件
  - 控制板：Nucleo-STM32G431RBT6
  - 功率板：X-Nucleo-IHM16M1
  - 电机： GBM2804H-100T
- 将JP8上的跳线冒拔除
  - 如果配置成功在JP8的2脚上可以测到精准参考电压2.5V



在MC SDK中使用VREFBUF

- 参照电机培训中介绍生成电机控制工程
- main.c中添加VREFBUF配置代码，本例配置为2.5V参考电压

main.c

```
115  /* Initialize interrupts */
116  MX_NVIC_Init();
117  /* USER CODE BEGIN 2 */
118
119  /* Configure internal VREFBUF voltage to 2.5V */
120  VREFBUF->CSR = 0x11;
121  /* Wait for the voltage reach to set value*/
122  while((VREFBUF->CSR & 0x08) == 0x00)
123  {
124    ;
125  }
126
127  /* USER CODE END 2 */
128
129  /* Infinite loop */
130  /* USER CODE BEGIN WHILE */
131  while (1)
```

life.augmented

修改参考电压

- 修改电压参考数据
  - parameters_conversion.h中修改为2.5V

parameters_conversion.h

```
25
26  #include "mc_math.h"
27  #include "parameters_conversion_g4xx.h"
28  #include "pmsm_motor_parameters.h"
29  #include "drive_parameters.h"
30  #include "power_stage_parameters.h"
31
32  //#define ADC_REFERENCE_VOLTAGE  3.30
33  /* Change Reference voltage value to 2.5V */
34  #define ADC_REFERENCE_VOLTAGE  2.50
35
```

life.augmented

最大电流数字量修改

- 修改最大电流数字量

$$New\_Value = \frac{Old\_Value * 3.3}{2.5}$$

- 修改最大退磁电流数字量
  - ID_DEMAG = -NOMINAL_CURRENT

最大电流数字量:

$$I_{digital} = \frac{I_{0-pk} * 32767 * Rshunt * OPGain * 2}{VREF}$$

pmsm_motor_parameters.h

```
36 /* When using Id = 0, NOMINAL_CURRENT is utilized to saturate the outp
37    PID for speed regulation (i.e. reference torque).
38    Transformation of real currents (A) into int16_t format must be do
39    formula:
40    Phase current (int16_t 0-to-peak) = (Phase current (A 0-to-peak)* 
41                                        *Amplifying network gain)/(MCU sup
42 */
43
44 #define NOMINAL_CURRENT          1504
45 #define MOTOR_MAX_SPEED_RPM      1572 /*!< Maximum rated speed  */
46 #define MOTOR_VOLTAGE_CONSTANT   5.0 /*!< Volts RMS ph-ph /kRPM */
47 #define ID_DEMAG                -1504 /*!< Demagnetization current */
```

pmsm_motor_parameters.h

```
36 /* When using Id = 0, NOMINAL_CURRENT is utilized to saturate the output
37    PID for speed regulation (i.e. reference torque).
38    Transformation of real currents (A) into int16_t format must be done
39    formula:
40    Phase current (int16_t 0-to-peak) = (Phase current (A 0-to-peak)* 327
41                                        *Amplifying network gain)/(MCU supply
42 */
43
44 //#define NOMINAL_CURRENT          1504
45 #define NOMINAL_CURRENT          1985 /* VREF = 2.5V, 1504*3.3/2.5*/
46 #define MOTOR_MAX_SPEED_RPM      1572 /*!< Maximum rated speed  */
47 #define MOTOR_VOLTAGE_CONSTANT   5.0 /*!< Volts RMS ph-ph /kRPM */
48 //#define ID_DEMAG                -1504 /*!< Demagnetization current */
49 #define ID_DEMAG                -1985
```

life.augmented

无传感启动电流修改

- 无传感启动阶段启动电流调整
  - 和上面计算相似，重新计算启动电流数字量
  - 本例中是无负载启动
  - 不做修改时启动电流变小

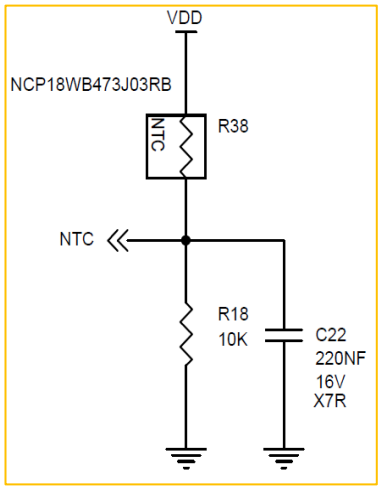
$$New\_Value = \frac{Old\_Value * 3.3}{2.5}$$

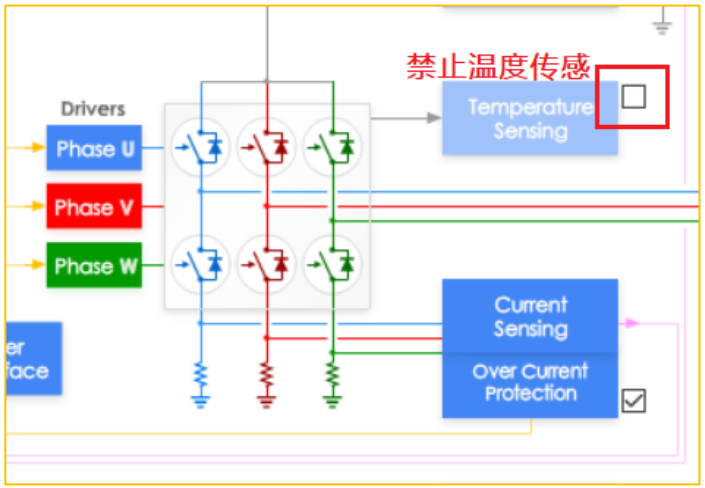
drive_parameters.h

```
164  /*******************************   START-UP PARAMETERS   ******
165
166  /* Phase 1 */
167  #define PHASE1_DURATION                1000 /*milliseconds */
168  #define PHASE1_FINAL_SPEED_UNIT         (0*SPEED_UNIT/_RPM)
169  #define PHASE1_FINAL_CURRENT           1504
170  /* Phase 2 */
171  #define PHASE2_DURATION                1164 /*milliseconds */
172  #define PHASE2_FINAL_SPEED_UNIT         (582*SPEED_UNIT/_RPM)
173  #define PHASE2_FINAL_CURRENT           1504
174  /* Phase 3 */
175  #define PHASE3_DURATION                0 /*milliseconds */
176  #define PHASE3_FINAL_SPEED_UNIT         (582*SPEED_UNIT/_RPM)
177  #define PHASE3_FINAL_CURRENT           1504
178  /* Phase 4 */
179  #define PHASE4_DURATION                0 /*milliseconds */
180  #define PHASE4_FINAL_SPEED_UNIT         (582*SPEED_UNIT/_RPM)
181  #define PHASE4_FINAL_CURRENT           1504
182  /* Phase 5 */
183  #define PHASE5_DURATION                0 /* milliseconds */
184  #define PHASE5_FINAL_SPEED_UNIT         (582*SPEED_UNIT/_RPM)
185  #define PHASE5_FINAL_CURRENT           1504
186
187  #define ENABLE_SL_ALGO_FROM_PHASE      2
```

注意事项

- 因为改小了参考电压，和参考电压相关参数需要调整
  - 电流采集满量程为0~2.5V，注意最大电流的电压值在这个范围内
  - 母线电压波动范围经过分压后电压需要在0~2.5V
    - 如果超过了则需要修改分压电阻
  - 温度传感器采集温度电压范围
    - 本例默认的温度传感系数超过计算范围，编译报错
      - 方法一：修改硬件分压，修改温度斜率
      - 方法二：暂时禁止温度传感功能
- 本例中只需要修改温度传感部分

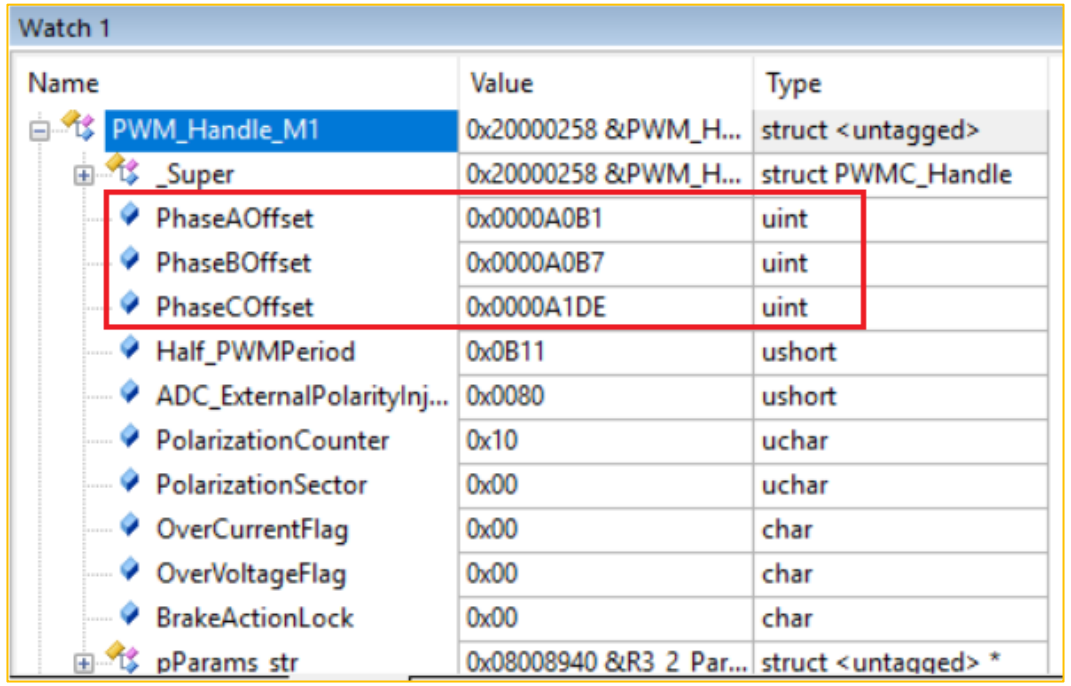




life.augmented

验证结果

- 编译下载后电机可正常运行
- 在调试窗口中可看到Offset电流数字量(左对齐)



Va _offset = 1.570V
Vb _offset = 1.570V
Vc _offset = 1.581V

$$Digital = \frac{V * 65536}{2.5V}$$

PhaseAOffset = 0xA0C4
PhaseBOffset = 0xA0C4
PhaseCOffset = 0xA1E4

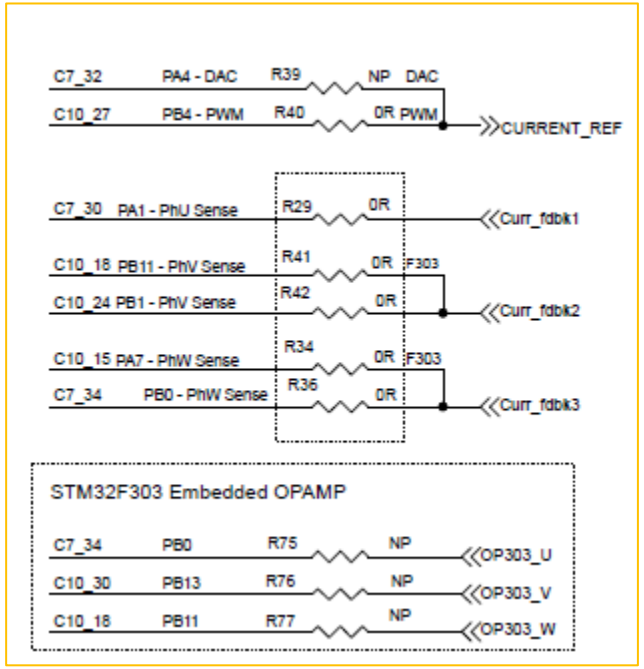
life.augmented

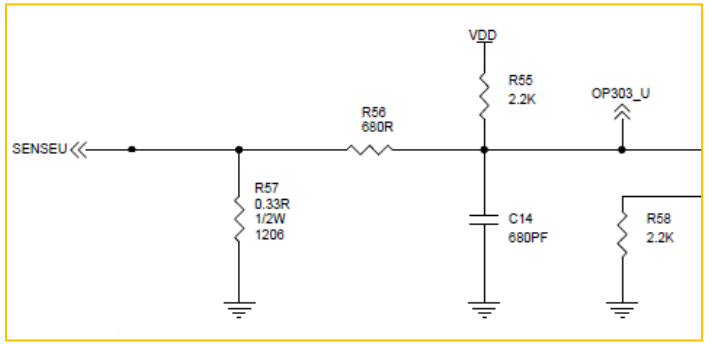
电流采样使用STM32G4内部运放

life.augmented

硬件修改

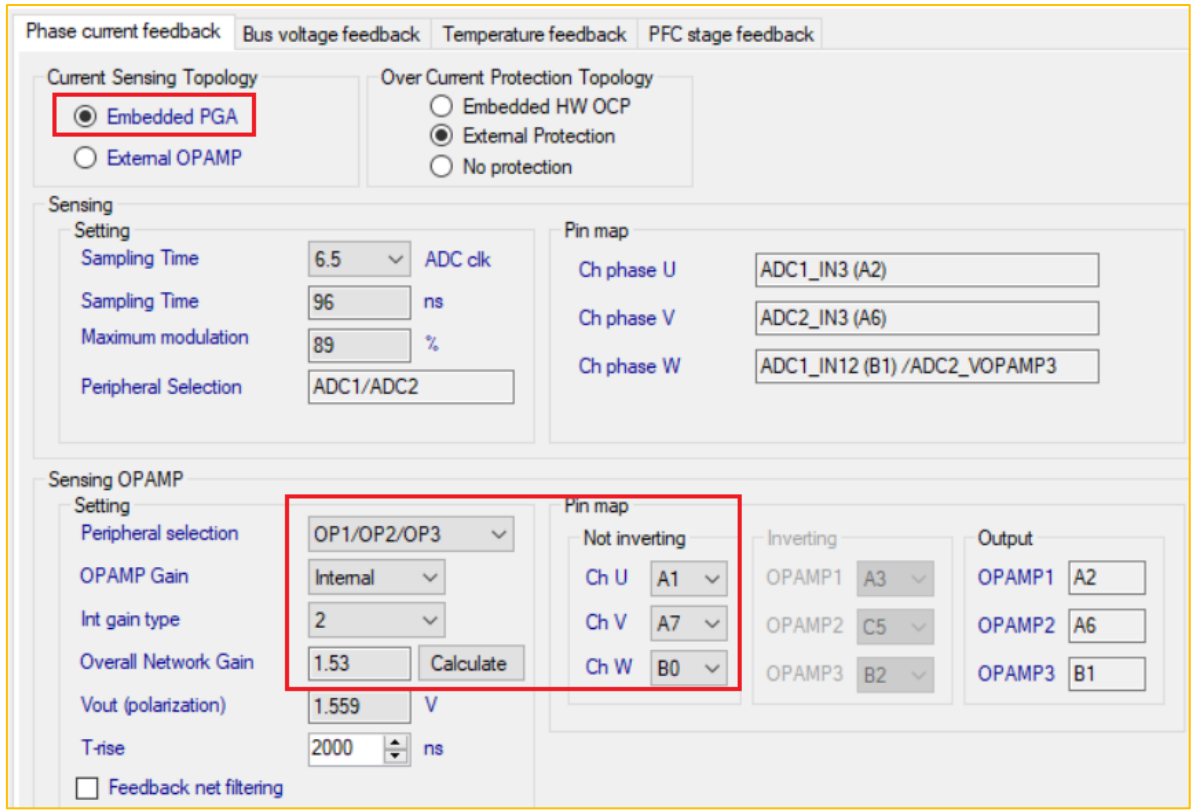
- 需要先对硬件电路进行修改，让采样电阻的电压接入运放引脚
- 如果使用的是Nucleo-G431RB+IHM16M1，修改如下：
  - 断开R29,R34,R36,R74,R75,R76,R77,R79
  - 连接R80
  - 连接OP303_U飞线到C7_30
  - 连接OP303_V飞线到C10_15
  - 连接OP303_W飞线到C7_34

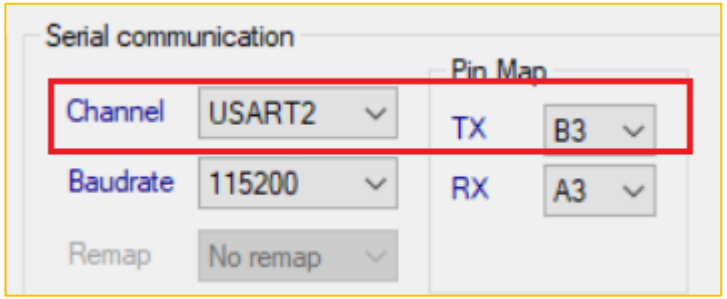




Workbench修改

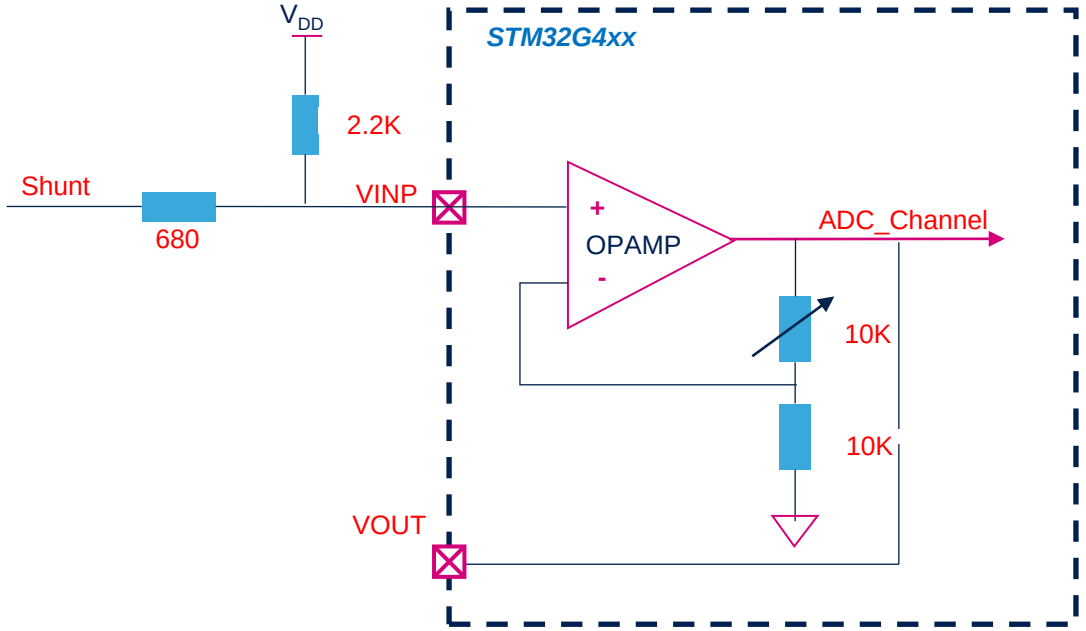
- 修改为内部PGA进行电流采样，注意放大倍数选择为2倍信号放大
  - 这边选择的是内部增益，如果使用外部增益则需要计算后填入
- 因为采样口占用了PA2，需要将串口修改为其他端口

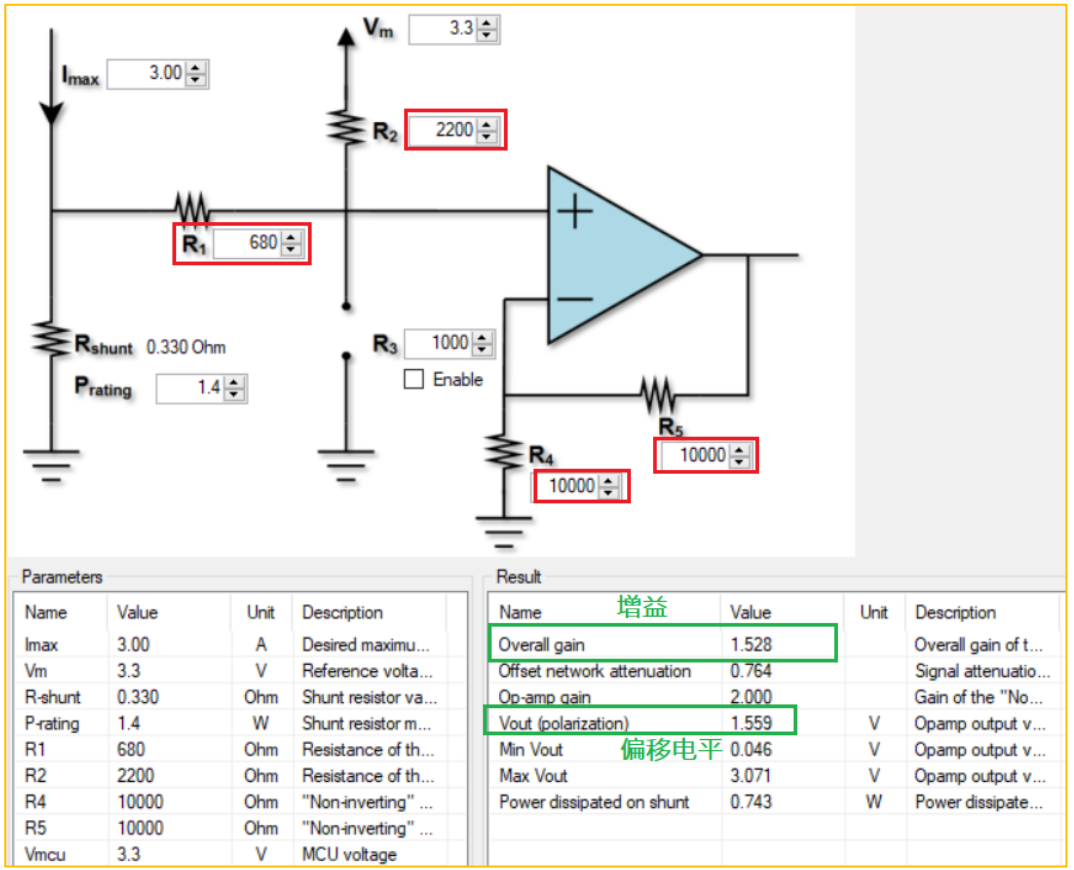




使用内部PGA放大倍数

- 配置完成后硬件连接示意图如下：
- 如果使用外部电路配置，需要计算外部电阻

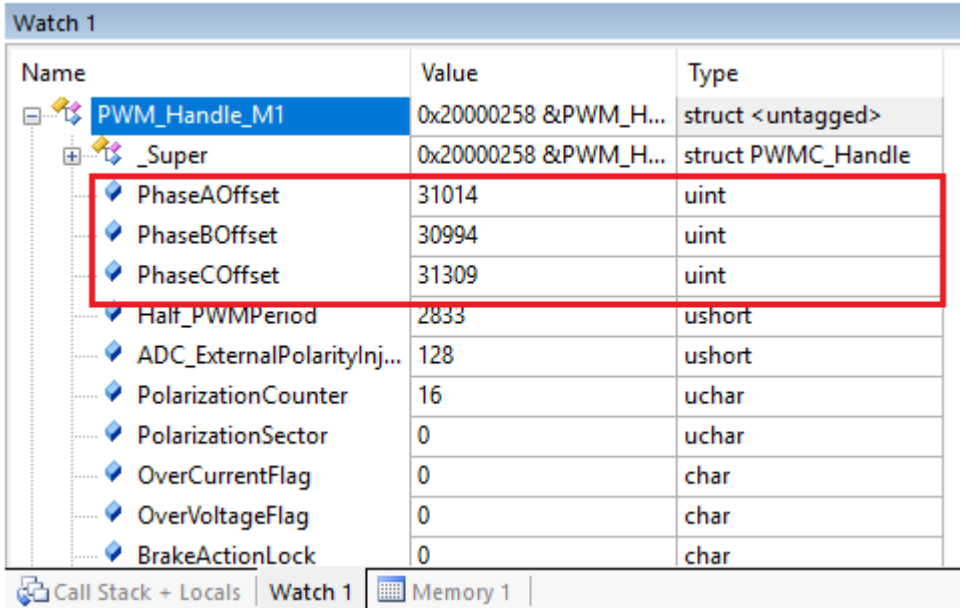




life.augmented

验证结果

- 生成工程，编译下载后运行电机
- 可在IDE观察窗口看到偏移电平数字值

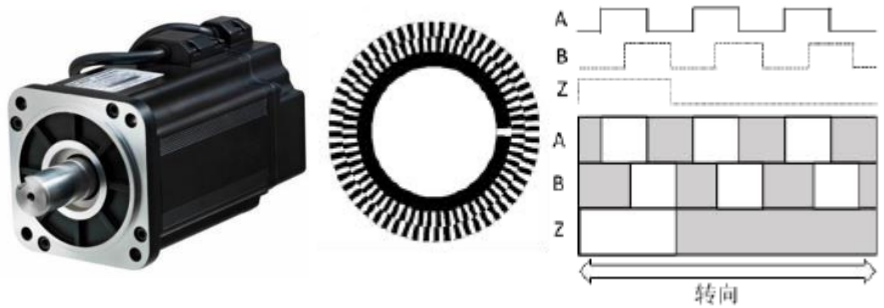


STM32G4的32-bit Timer应用于编码器电机应用

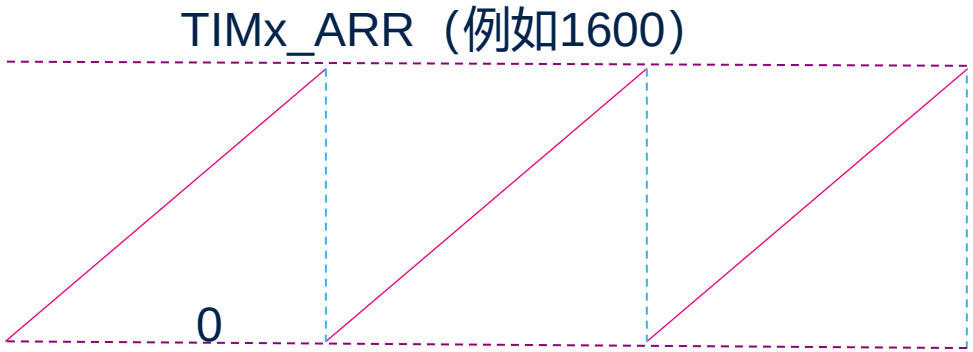
life.augmented

当前电机Encoder模式

- MC SDK V5.4相对编码器使用A/B线
- Timer工作在Encoder从模式
- TIMx_ARR为编码器分辨率的4倍
- 当前存在数据突变问题，对于伺服类应用有限制

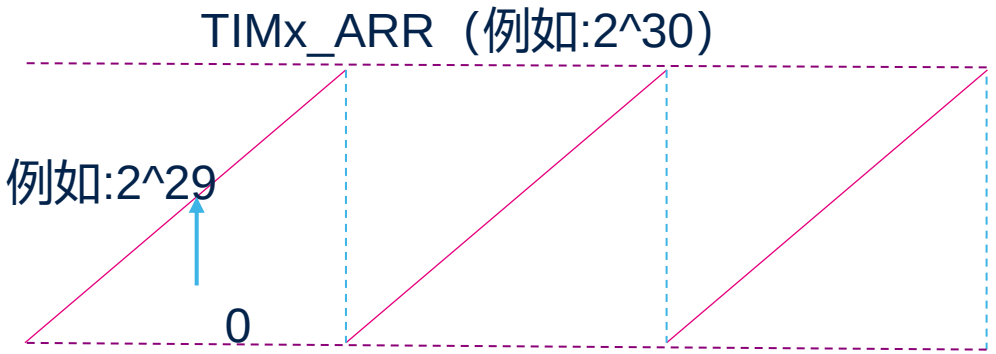


编码器分辨率（例如400）



可考虑的改进

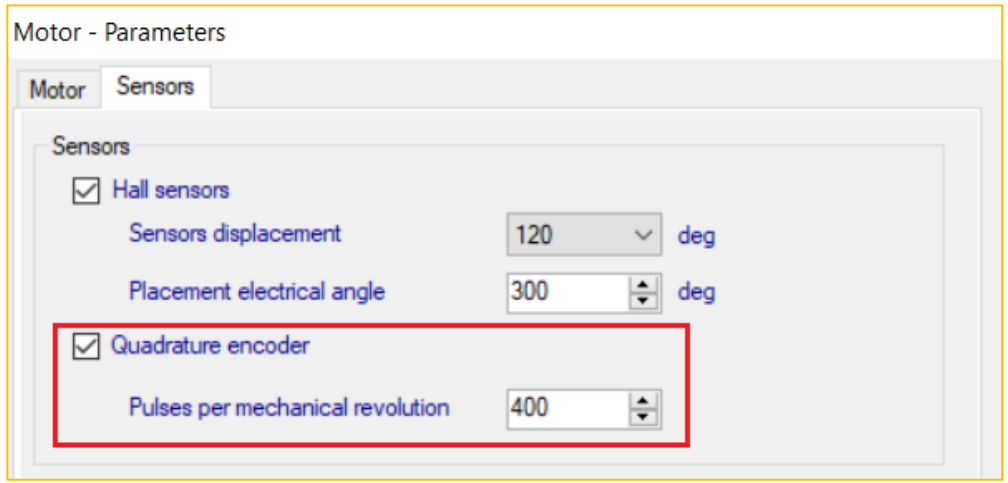
- 如果我们充分利用了32-bit Timer进行Encoder计数
- 例如我们设定30-bit的TIMx_ARR寄存器
- 设定起始计数TIMx_CNT为29-bit
- 这样我们需要经过2^29才有一次数据突变
  - 因为2^29+2^29 = 2^30
  - 如果是前面400线的电机，需要电机运行335544圈才有一次数据突变！
  - 假如电机是1000RPM运行，那么连续同一个方向转接近5.6个小时才有数据突变
- 伺服应用将得到极大改善

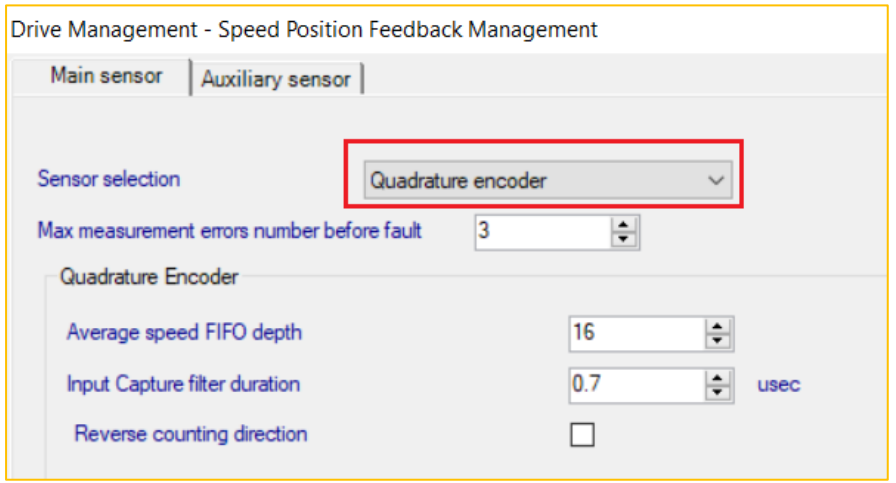


life.augmented

前期准备

- 本例采用P-NUCLEO-IHM03为测试硬件
- 电机需要更换为有编码器的电机
  - 控制板：Nucleo-STM32G431RBT6
  - 功率板：X-Nucleo-IHM16M1
  - 电机： Shinano-LA052-080E3NL1
- 请参考电机控制培训的《Encoder使用案例》， TIM2为Encoder模式
  - 电机Encoder模式的正常运行





重新定义编码器函数

- MC SDK V5.4中库函数大多数定义为__*weak*属性
- 方便我们重新定义函数
- 可以不破坏结构下重新自定义函数
- 程序调用自定义函数，而忽略__*weak*属性的同名函数
  - 比如我们定义encoder_sensor_improve.c 中为自定义函数文件
  - 库文件原文件名为encoder_speed_pos_fdbk.c

```
__weak void ENC_Init( ENCODER_Handle_t * pHandle )
{

  TIM_TypeDef * TIMx = pHandle->TIMx;
  uint8_t BufferSize;
  uint8_t Index;
```

```
void ENC_Init( ENCODER_Handle_t * pHandle )
{

  TIM_TypeDef * TIMx = pHandle->TIMx;
  uint8_t BufferSize;
  uint8_t Index;
  uint32_t wtemp1;
```

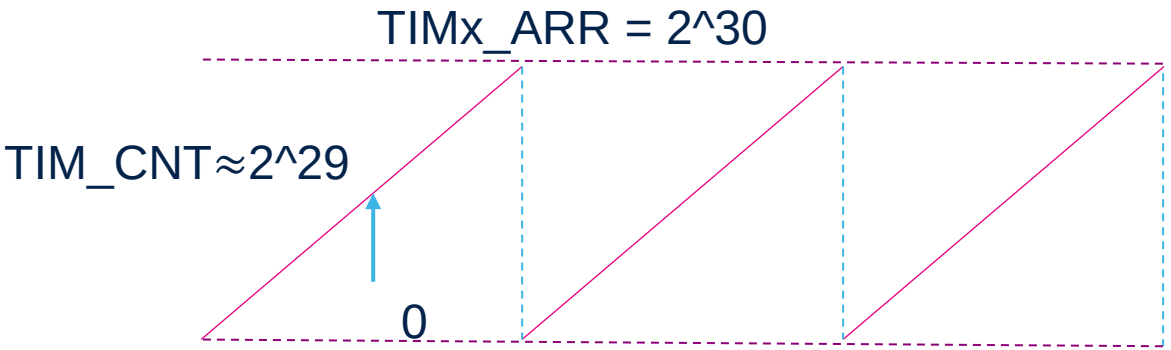
初始化TIM2_CNT

- 重新配置TIM_ARR为2^30
- 重新配置Counter初始数据在2^29附近
  - 配置电角度为0的CNT数据

```
58  void ENC_Init( ENCODER_Handle_t * pHandle )
59  {
60
61    TIM_TypeDef * TIMx = pHandle->TIMx;
62    uint8_t BufferSize;
63    uint8_t Index;
64    uint32_t wtempl;
65
66
67  #ifdef TIM_CNT_UIFCPY
68    LL_TIM_EnableUIFRemap ( TIMx );
69  #define ENC_MAX_OVERFLOW_NB      ((uint16_t)2048) /* 2^11*/
70  #else
71  #define ENC_MAX_OVERFLOW_NB      (1)
72  #endif
73
74    /* Set CNT initial value to near 2^29 */
75    wtempl = (uint32_t)MID_ENCODER_COUNTER/(uint32_t)( pHandle->PulseNumber );
76    wtempl *= (uint32_t)( pHandle->PulseNumber );
77
78    /* Reset counter */
79    LL_TIM_SetCounter ( TIMx, wtempl );
80
81    /* Set ARR to 2^30 */
82    LL_TIM_SetAutoReload( TIMx, MAX_ENCODER_COUNTER );
83
```

配置TIM_CNT

配置TIM_ARR



life.augmented

修改计算电角度方式

- 首先机械角度限制到360度之内
- 机械角度数字化
- 函数返回电角度

$$mecAngle = \frac{(CNT\%Pulse) * 65536}{Pulse}$$

$$elAngle = mecAngle * Poles$$

*注：Pulse = 编码器分辨率*4*

```
130  int16_t ENC_CalcAngle( ENCODER_Handle_t * pHandle )
131  {
132    uint32_t wtempl;
133    int16_t elAngle;  /* s16degree format */
134    int16_t mecAngle; /* s16degree format */
135
136    /* Data limit into one mechanical angle */
137    wtempl = ( uint32_t )( LL_TIM_GetCounter( pHandle->TIMx ) & 0xffffffff )
138                         % (pHandle->PulseNumber);
139
140    /* Digital mechanical angle */
141    mecAngle = ( int16_t )( (int32_t)wtempl * 65536 / (pHandle->PulseNumber) );
142
143    pHandle->_Super.hMecAngle = mecAngle;
144
145    /* Electrical angle = Mec_Angle*Poles */
146    elAngle = (mecAngle * pHandle->_Super.bElToMecRatio);
147    pHandle->_Super.hElAngle = elAngle;
148
149    /*Returns rotor electrical angle*/
150    return ( elAngle );
151  }
```

life.augmented

修改计算平均速度的方法

- 修改ENC_CalcAvrgMecSpeedUnit函数
- 判断当前电机运行方向，正向还是反向
- 通过编码器速度环路中前后两次读数计算速度
- 计算平均速度
  - *详细修改代码可参考HandsOn文件*

$$RPM = \frac{(CNT - CNT_{pre}) * f_{ASR} * 60}{Pulse}$$

CNT —— $Encoder$计数现值

CNT_{pre} —— $Encoder$计数前值

f_{ASR} —— 速度环调整频率

```
bool ENC_CalcAvrgMecSpeedUnit( ENCODER_Handle_t * pHandle, int16_t * pMecSpeedUnit )
{
  TIM_TypeDef * TIMx = pHandle->TIMx;
  int32_t wtempl;

  bool bReliability = true;

  /* Get now counter */
  Encoder_Counter_Now = LL_TIM_GetCounter ( TIMx );

  /* Judge the direction and */
  if(Encoder_Counter_Now > Encoder_Counter_Pre)
  {
    Encoder_Delta_Counter = Encoder_Counter_Now - Encoder_Counter_Pre;
    /* if delta value bigger than limit, means an update happened */
    if(Encoder_Delta_Counter > UPDATE_DELTA_LIMIT)
    {
```

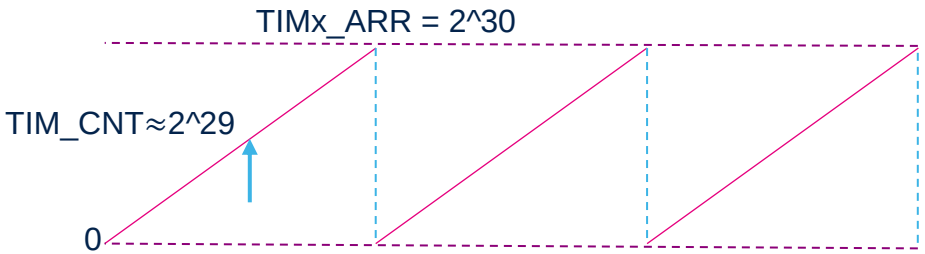
life.augmented

设定计数值在中值附近

- 设定电角度时，增加计数中值偏移值
- 正常计算电角度对应Count数据
- 计算新的TIMER的CNT数据

$$Count = \frac{MecAngle\_Digital * Pulse}{65536}$$

$$TIMx\_CNT = Count + Mid\_cycle\_counter$$

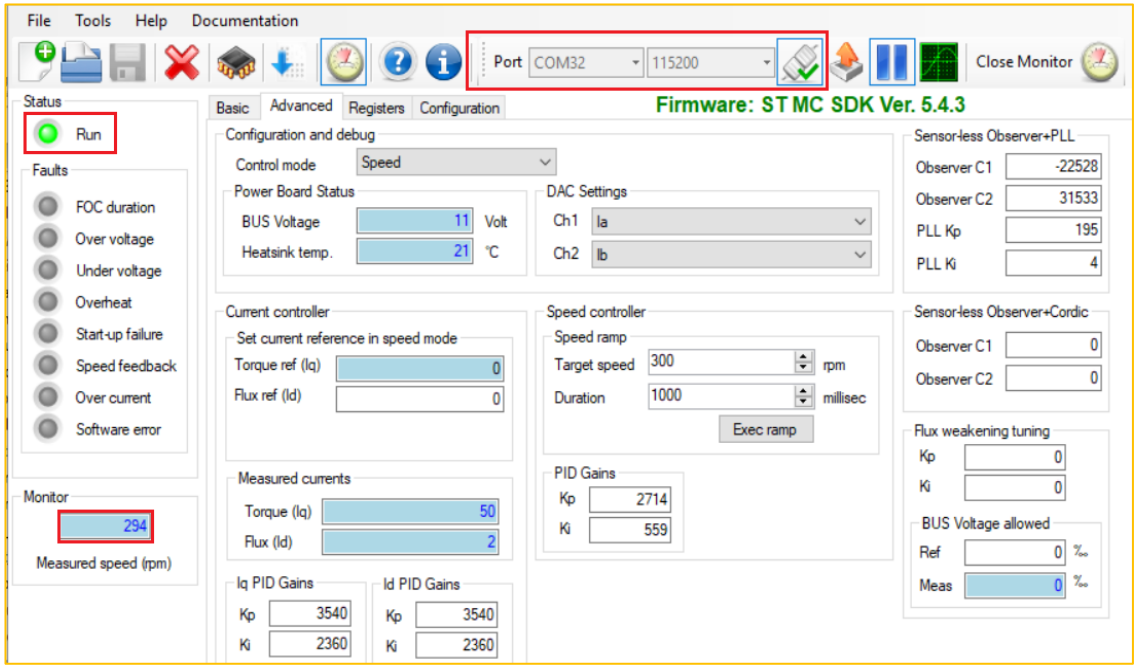


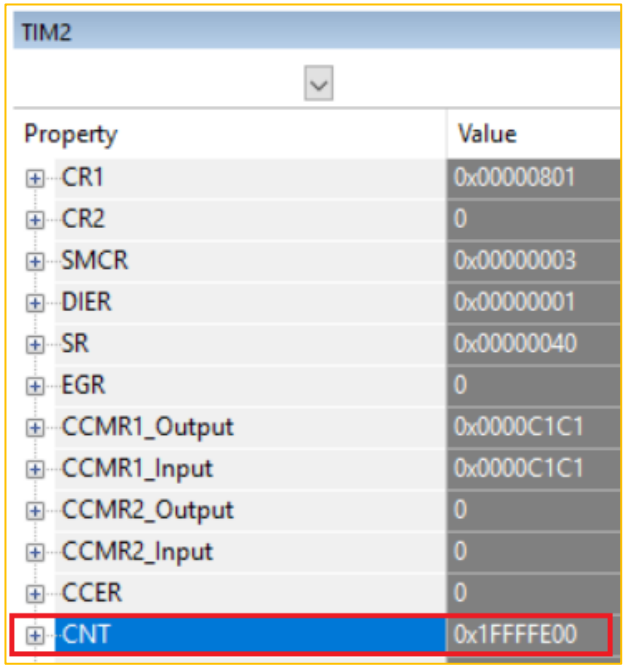

```
void ENC_SetMecAngle( ENCODER_Handle_t * pHandle, int16_t hMecAngle )
{
  TIM_TypeDef * TIMx = pHandle->TIMx;

  uint16_t hAngleCounts;
  uint16_t hMecAngleuint;
  uint32_t wtemp;

  pHandle->_Super.hMecAngle = hMecAngle;
  pHandle->_Super.hElAngle = hMecAngle * pHandle->_Super.bElToMecRatio;
  if ( hMecAngle < 0 )
  {
    hMecAngle *= -1;
    hMecAngleuint = 65535u - ( uint16_t )hMecAngle;
  }
  else
  {
    hMecAngleuint = ( uint16_t )hMecAngle;
  }

  hAngleCounts = ( uint16_t )( ( ( uint32_t )hMecAngleuint *
                               ( uint32_t )pHandle->PulseNumber ) / 65535u );

  wtemp = (uint32_t)MID_ENCODER_COUNTER/(uint32_t)( pHandle->PulseNumber );
  wtemp *= (uint32_t)( pHandle->PulseNumber );

  TIMx->CNT = ( uint32_t )( hAngleCounts + wtemp);

}
```

life.augmented

验证结果

- 修改完成后，编译工程并下载程序到单片机中运行
- 可以任意设定速度，在编码器模式下正常调节
- 可以仿真窗口上看到TIM_CNT初始数据在2^29(0x20000000)附近





总结

- 本篇章结合ST电机控制生态系统进行试验
- 将关键程序(如电流环，观测器)放在CCM SRAM中可大幅提高运行速度
- 使用VREFBUF可以让ADC采样更精准
- 可以直接使用STM32G4内部运放用于电机控制
- 32-bit Timer利于编码器应用
- STM32G4内部其他外设逐步被发掘使用中

Thank you

life.augmented